

Structural Properties of Compressed Tries

1 Notation

Symbols

Let S be the set of stored strings. We use:

$$s = |S|, \quad d = |\Sigma|, \quad T = \text{compressed trie built from } S.$$

2 Degree Property

Internal-Node Degree

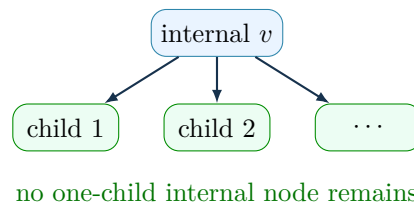
In a compressed trie, every internal node has

at least 2 children

and at most

d children.

The lower bound holds because one-child internal nodes have been compressed away. The upper bound holds because there is at most one outgoing branch per alphabet symbol.



3 External Nodes

If each stored string is represented by one leaf, then a compressed trie has

s external nodes.

Compression changes internal structure, but it does not change the fact that every stored string needs one endpoint.

4 Number of Nodes

Node Bound

For a compressed trie storing s strings,

$$|T| = \mathcal{O}(s).$$

The key reason is that every internal node branches. With s leaves and no one-child internal nodes, the number of internal branching nodes is linear in s .

5 String Labels as References

Compressed tries should not copy long substrings into every edge label. Instead, an edge label can be represented by:

- a substring reference;
- a start index;
- an end index;
- a pointer into the original text or dictionary.

Index View

A compressed trie is best treated as an index. Edges identify substrings without repeatedly storing full copies of those substrings.

Compact Edge Label

Instead of storing a full edge string, store

$$(i, j, k)$$

meaning: use substring $S[i][j..k]$ from the original string collection.

This keeps the tree small: the original strings stay in one array, and trie edges only point into that array.

6 Standard vs Compressed Trie

Feature	Standard trie	Compressed trie
Edge unit	One character	One or more characters
One-child internal nodes	Allowed	Removed
Minimum internal degree	1	2
Node count	$\mathcal{O}(n)$	$\mathcal{O}(s)$
Main benefit	Simple traversal	Reduced structural redundancy

Here

$$n = \sum_{w \in S} |w|, \quad s = |S|.$$